

STRINGS, OUTPUT, AND FIRST PROGRAM

Making programs talk like humans

```
project_name = "Network Modernization"  
completed_tasks = 18  
total_tasks = 24  
percent_complete = completed_tasks /  
total_tasks * 100
```

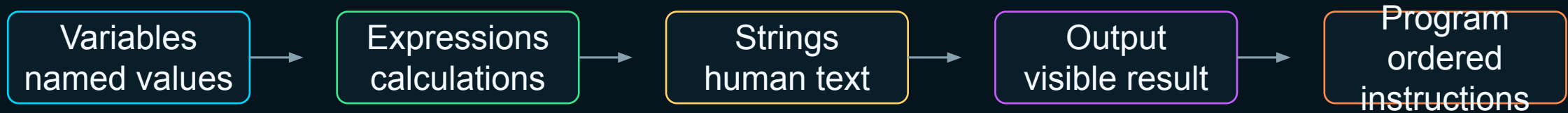
```
print(f"Project: {project_name}")  
print(f"Completed tasks:  
{completed_tasks}")  
print(f"Progress: {percent_complete:.1f}%")
```

```
Project: Network Modernization  
Completed tasks: 18  
Progress: 75.0%
```

30 min instruction • 30 min guided lab • 30 min applied challenge

From values to a program

Last block calculated values. This block makes those values readable and builds the first complete mini-program.



Goal: write code that stores facts, calculates new facts, and prints a clean summary.

String literals: text inside quotes

A string is how Python stores text such as names, labels, filenames, and messages.

```
project_name = "Network  
Modernization"  
file_name =  
"equipment_records.csv"  
status = "active"
```

"Network
Modernization"

The quotes are syntax. The text
inside is data.

Strings can contain letters, digits, spaces, punctuation, and emoji — but Python treats them as text.

Quotation marks: choose a pair and close it

Python accepts single or double quotes. Most beginner errors come from forgetting the closing quote.

```
name = "Mike"  
name = 'Mike'  
message = "Don't panic"
```

```
print("Hello)  
# SyntaxError: unterminated  
string literal
```

opening quote

text data

closing quote

Good habit: use double quotes most of the time, then be consistent.

Combining strings: three common approaches

Python gives several ways to build output. Some are beginner-friendly; some are more flexible.

```
print("Project: " +  
project_name)
```

```
print("Project:",  
project_name)
```

```
print(f"Project:  
{project_name}")
```

+ joins strings only

comma prints pieces

f-string embeds values

In this course, use f-strings for clean formatted output.

Converting values to strings

Python will not always guess how to combine text and numbers.

```
completed_tasks = 18
```

```
print("Completed: " +  
      completed_tasks)
```

TypeError
can only concatenate str to str

```
print("Completed: " +  
      str(completed_tasks))  
print(f"Completed: {completed_tasks}")
```

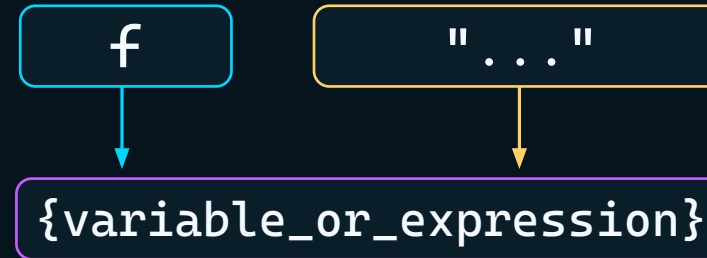
Completed: 18

Best beginner habit: f-strings usually avoid manual str() conversion.

Formatted string literals: f-strings

Put f before the quotes. Put variable names or expressions inside braces.

```
name = "Mike"  
years = 8  
  
print(f"{name} has {years} years of  
service.")
```



Output: Mike has 8 years of service.

Formatting numbers for readable output

Computers may calculate long decimals. Humans usually need rounded display values.

```
percent_complete = 18 / 24 * 100  
  
print(percent_complete)  
print(f"Progress: {percent_complete:.1f}%")
```

75.0

Progress: 75.0%

`:.1f`

means “show one digit after the decimal point”

Formatting changes the display, not necessarily the underlying stored value.

First mini-program: project summary

A complete beginner program can store facts, calculate progress, and print a readable summary.

```
project_name = "Network Modernization"
completed_tasks = 18
total_tasks = 24
percent_complete = completed_tasks /
total_tasks * 100

print(f"Project: {project_name}")
print(f"Completed tasks: {completed_tasks}
of {total_tasks}")
print(f"Progress: {percent_complete:.1f}%")
```

Project: Network Modernization
Completed tasks: 18 of 24
Progress: 75.0%

facts

calculatio
n

output

Readable output is a design choice

A correct answer is not always a useful answer. Labels and units matter.

```
print(percent_complete)
```

75.0



```
print(f"Progress:  
{percent_complete:.1f}%")
```

Progress: 75.0%

When output leaves your screen and goes to a report, log, email, or dashboard, context matters.

Simple debugging: errors are clues, not insults

Python usually tells you the kind of problem and where it noticed the issue.

SyntaxError

Python cannot parse the
code

NameError

a name was used before
it existed

TypeError

an operation did not
make sense for the value
types

Debugging habit: read the last line, check the spelling, check the types, then check the line above.

SyntaxError: Python cannot parse the code

A syntax error is like a grammar problem in the programming language.

```
print("Project status")  
  
# missing closing quote
```

```
File "program.py", line 1  
    print("Project status")  
        ^  
SyntaxError: unterminated string  
literal
```

```
print("Project status")
```

Fix: close the quote and parentheses

NameError: Python does not know that name

A name must exist before Python can use it. Spelling and capitalization matter.

```
project_name = "Network  
Modernization"  
  
print(projectname)
```

```
NameError  
name 'projectname' is not defined
```

```
print(project_name)
```

Fix: match the variable name exactly

TypeError: the operation does not fit the values

Types describe what kind of value Python is handling. Some operations only work on compatible types.

```
tasks = 18  
  
print("Completed: " + tasks)
```

TypeError
can only concatenate str to str

```
print(f"Completed: {tasks}")
```

Fix: use an f-string or convert with
str()

Traceback habit: read the last line first

The final line often names the error type and gives the most useful clue.

```
Traceback (most recent call last):  
  File "program.py", line 5, in <module>  
    print(f"Progress: {percent_complete:.1f}%")  
NameError: name 'percent_complete' is not defined
```

1. Error type

2. Message

3. Line number

4. Nearby code

Do not panic-scroll. Start with the last line, then work upward.

Guided exercise: build a project summary program

Students create a short program with variables, calculations, comments, and formatted output.

```
# Project summary
project_name = "Network
Modernization"
team_size = 5
completed_tasks = 18
total_tasks = 24
is_active = True
```

```
Progress: 75.0%
Status active: True
```

- print values, print types
- print X of Y completed tasks
- print progress

Lab 1: print variables and types

Start with values. Then use `type()` to ask Python what each value is.

```
project_name = "Network Modernization"  
team_size = 5  
is_active = True  
  
print(project_name)  
print(type(project_name))  
print(type(team_size))  
print(type(is_active))
```

<class 'str'>

<class 'int'>

<class 'bool'>

Question: which values are text, whole numbers, and true-or-false conditions?

Lab 2: add calculations and formatted output

Calculate progress, then print a complete project summary.

```
completed_tasks = 18
total_tasks = 24
percent_complete = completed_tasks /
total_tasks * 100

print(f"Completed tasks: {completed_tasks}
of {total_tasks}")
print(f"Progress: {percent_complete:.1f}%")
```

```
Completed tasks: 18 of 24
Progress: 75.0%
```

Predict first. Run second. Explain third.

Lab 3: break it on purpose, then fix it

Intentional errors make error messages less scary.

Remove a quote
→ `SyntaxError`

Misspell a variable
→ `NameError`

Use `+` with text and int
→ `TypeError`

For each error: read the last line, identify the error type, fix one thing, run again.

Debugging checklist:

1. What kind of error is it?
2. What line did Python point to?
3. What name, quote, parenthesis, or type is involved?
4. What is the smallest fix to try?

Build a data-storage calculator

Use strings, numbers, calculations, f-strings, and type() in one small program.

```
file_name = "equipment_records.csv"
size_bytes = 2_500_000

size_kilobytes = size_bytes / 1_000
size_megabytes = size_bytes / 1_000_000
```

```
print(f"File: {file_name}")
print(f"Bytes: {size_bytes}")
print(f"Kilobytes: {size_kilobytes}")
print(f"Megabytes: {size_megabytes}")
```

```
File: equipment_records.csv
Bytes: 2500000
Kilobytes: 2500.0
Megabytes: 2.5
```

Challenge extras: make output cleaner

Small improvements make the program feel more professional.

```
print(f"Kilobytes: {size_kilobytes:,.1f} KB")
print(f"Megabytes: {size_megabytes:,.2f} MB")

print(type(file_name))
print(type(size_bytes))
print(type(size_megabytes))
```

```
Kilobytes: 2,500.0 KB
Megabytes: 2.50 MB
```

`:,.1f`

commas plus one decimal
place

`:,.2f`

commas plus two decimal
places

Formatting makes output easier to scan without changing the original calculation.

End-of-block check

Student

What is a string literal?

What does an f-string do?

Why might `print(75.0)` be less useful than `print("Progress: 75.0%")`?

What are `SyntaxError`, `NameError`, and `TypeError`?

Why is the final line of an error message useful?

What does Python do before the processor ultimately executes instructions?

Good answer style: clear, practical, and connected to what the code actually did.